How Large Language Models Work

Video

Please visit the textbook on a web or mobile device to view video content.

LLM Visualization

At its core, a large language model is a sophisticated statistical machine designed to predict the most probable next word (or more accurately, "token") in a sequence, given the preceding context. It doesn't "understand" language in a human sense; rather, it identifies intricate patterns and relationships between words learned from vast amounts of text data.

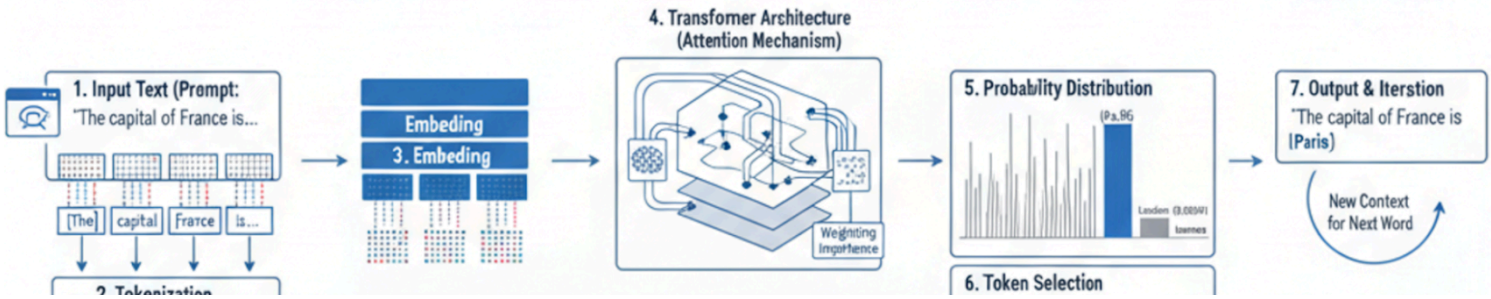
Here's a breakdown of key terminology in sequence to demonstrate how LLMs work :

1. **Input Text (Prompt):** The user provides an initial piece of text, often called a "prompt." This text is the context the LLM will use to make its prediction.
2. **Tokenization:** The first step is to break down the input text into smaller units called "tokens." A token can be a whole word, a sub-word unit (like "un-" or "-ing"), punctuation, or even a single character for some models.

This allows the model to handle a wider vocabulary and common prefixes/suffixes.

3. **Embedding:** Each token is then converted into a numerical representation called an "embedding vector." These vectors are multi-dimensional arrays where similar words (e.g., "cat" and "kitten") have similar vector representations in the embedding space. This process allows the model to work with mathematical operations.
4. **Transformer Architecture (Attention Mechanism):** The heart of modern LLMs is often the Transformer architecture, particularly its "attention mechanism." This mechanism allows the model to weigh the importance of different tokens in the input sequence when predicting the next one. For example, if the prompt is "The quick brown fox jumps over the lazy...", the model will pay more "attention" to "fox" and "jumps" than to "the" or "brown" when predicting "dog" or "cat." It processes these embeddings through many layers, identifying complex relationships and context.
5. **Probability Distribution:** After processing the input through its many layers, the LLM outputs a probability distribution over its entire vocabulary for the *next possible token*. This means it assigns a likelihood to every single word it knows. For instance, given "The capital of France is", it might assign a very high probability to "Paris," a moderate probability to "London" (if there's a slight chance of error or ambiguity in context), and a very low probability to "banana."
6. **Token Selection:**
    1. **Greedy Sampling:** The simplest method is to just pick the token with the highest probability.
    2. **Sampling with Temperature:** More commonly, the model uses a sampling method that introduces a bit of randomness ("temperature"). A higher temperature makes the output more creative and diverse by increasing the chances of less probable tokens being selected, while a lower temperature makes it more deterministic and focused on the highest probability tokens.
    3. **Top-K / Top-P (Nucleus) Sampling:** These methods narrow down the selection pool to the top 'K' most probable tokens or to a subset of tokens whose cumulative probability exceeds a certain threshold 'P', and then sample from that reduced set.
7. **Output & Iteration:** The selected token is then appended to the input sequence, and the entire process repeats. The newly predicted token becomes part of the new "context" for predicting the *next* token. This iterative prediction is how LLMs generate coherent sentences, paragraphs, and longer texts.

Essentially, an LLM is a very sophisticated autocomplete system that has learned from a massive amount of human-generated text to predict what word should come next with remarkable accuracy and fluency.


4. Transformer Architecture
(Attention Mechanism)
1. Input Text (Prompt:
"The capital of France is...
[The]
capital
France
Is...
Embeding
3. Embeding
Weighting
Importance
5. Probability Distribution
6. Token Selection
7. Output & Iterstion
"The capital of France is
(Paris)
New Context
for Next Word